

Q2 – Gaussian Elimination Using CUDA

Introduction

For Q2, we implemented Gaussian Elimination to solve the system of linear equations:

$$\mathbf{AX} = \mathbf{B}$$

We implemented the algorithm in two ways:

1. Sequential implementation using the CPU
2. Parallel implementation using CUDA on the GPU

The main purpose of this question is to compare the performance of the CPU and GPU implementations and calculate the speedup obtained using CUDA.

Problem Description

We tested Gaussian Elimination using two different matrix sizes:

- $n = 1000$
- $n = 2000$

For each matrix size, the program generates matrix $\mathbf{A}$ and vector $\mathbf{B}$, and then solves for $\mathbf{X}$.

The matrix is generated in a way that makes the system suitable for Gaussian Elimination.

Implementations

Sequential CPU Implementation

In the sequential implementation, Gaussian Elimination is performed using the CPU.

The CPU performs the forward elimination step row by row and then performs back substitution to obtain the solution vector.

The execution time of the sequential implementation is used as the reference time for calculating CUDA speedup.

CUDA Implementation

In the CUDA implementation, the independent calculations during Gaussian Elimination are performed in parallel using GPU threads.

For each pivot step, CUDA threads update different elements of the matrix simultaneously.

After forward elimination is completed, back substitution is performed to calculate the final solution.

CUDA Thread Configurations

We tested three different CUDA block configurations:

- $8 \times 8 = 64$ threads
- $16 \times 16 = 256$ threads
- $32 \times 32 = 1024$ threads

All three configurations were tested for both:

- $n = 1000$
- $n = 2000$

This allows us to compare how different numbers of threads affect the performance of the CUDA implementation.

Number of Runs

Each configuration was executed **5 times**.

For every run, we recorded the execution time.

The average execution time was calculated using:

$$\text{Average Time} = (\text{Run 1} + \text{Run 2} + \text{Run 3} + \text{Run 4} + \text{Run 5}) / 5$$

Running each configuration multiple times gives us a more reliable measurement instead of depending on only one run.

Timing

We used the `gettimeofday()` function to measure the execution time.

The time was recorded before and after the computation.

The difference between these two values gives the execution time of the program.

The execution time is recorded in milliseconds.

Speedup

The CUDA speedup is calculated using:

$$\text{Speedup} = \text{Sequential Average Time} / \text{CUDA Average Time}$$

For example, if the sequential implementation takes 1000 ms and CUDA takes 250 ms:

$$\text{Speedup} = 1000 / 250 = 4\times$$

This means the CUDA implementation is 4 times faster for that particular configuration.

Correctness Check

We did not only compare execution times. We also checked whether the CUDA implementation produces a correct solution.

The residual is calculated using:

$$\text{Residual} = \max |AX - B|$$

A very small residual means that the calculated solution satisfies the original equation correctly.

We also compare the solution obtained from the CPU and CUDA implementations.

The maximum difference is calculated as:

$$\text{Max Difference} = \max |X_{\text{CPU}} - X_{\text{CUDA}}|$$

A very small maximum difference means that both implementations produce almost the same solution.

Source Code

The main CUDA source code is:

`q2_gaussian.cu`

Compilation

We used NVIDIA's CUDA compiler `nvcc`.

The program can be compiled using:

```
nvcc -O3 q2_gaussian.cu -o q2_gaussian
```

Running the Program

After compiling the program, run:

```
./q2_gaussian
```

The program performs the required experiments for:

- $n = 1000$
- $n = 2000$
- 8×8 threads
- 16×16 threads

- 32×32 threads

Each configuration is executed 5 times.

Output

After running the program, the experimental results are stored in:

`q2_results.csv`

The CSV file contains the results for all runs.

The information includes:

- Matrix size
- Method
- Number of threads
- Number of blocks
- Run number
- Execution time
- Average execution time
- Speedup
- Residual
- Maximum solution difference

A summarized file is also generated:

`q2_summary.csv`

This file contains the main CUDA results used for plotting.

Graphs

We created two graphs to analyze the CUDA performance.

Execution Time Graph

The execution-time graph is saved as:

`q2_execution_time.png`

This graph compares the average execution time for the different CUDA thread configurations.

Speedup Graph

The speedup graph is saved as:

q2_speedup.png

This graph shows the speedup of CUDA compared with the sequential CPU implementation.

Files Included

The following files are used for Q2:

q2_gaussian.cu

Main CUDA implementation.

q2_results.csv

Contains results from all experimental runs.

q2_summary.csv

Contains summarized CUDA results.

q2_time.gnuplot

GNUplot script for generating the execution-time graph.

q2_execution_time.png

Execution-time graph.

q2_speedup.gnuplot

GNUplot script for generating the speedup graph.

q2_speedup.png

Speedup graph.

Environment Used

The implementation was tested using:

- Google Colab
- NVIDIA CUDA-enabled GPU
- CUDA C++
- nvcc compiler
- GNUplot
- Excel for organizing the results

Conclusion

In Q2, we implemented Gaussian Elimination using both CPU and CUDA.

We tested two matrix sizes and three different CUDA thread configurations. Each configuration was executed five times and the average execution time was calculated.

We then compared the CUDA execution time with the sequential CPU execution time and calculated the speedup.

We also checked the correctness of the CUDA implementation using the residual and the maximum difference between the CPU and CUDA solutions.

The graphs and Excel results help us understand how GPU parallelization and different CUDA thread configurations affect the performance of Gaussian Elimination.